

Agentic Calendar Assistant

Training a Tool-Calling LLM on Consumer Hardware

Full Pipeline — Data Generation • SFT • RL

NVIDIA TITAN X Pascal • 12 GB VRAM • Qwen 2.5-1.5B

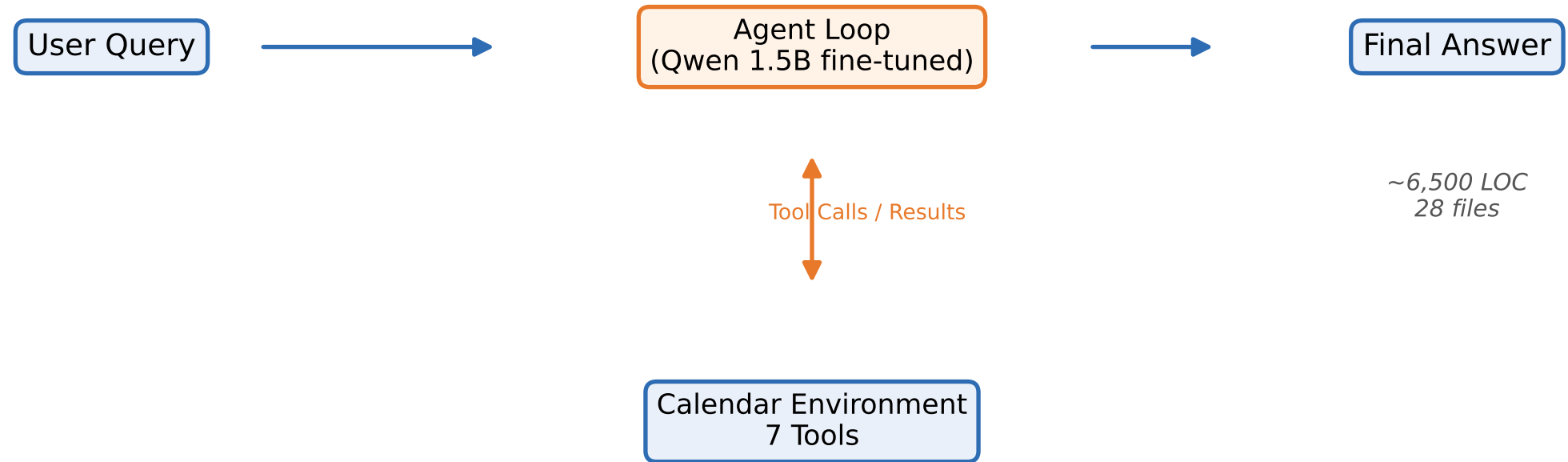
Problem Statement

- Calendar management requires multi-step reasoning + tool use
- Commercial APIs (GPT-4, Gemini) work but are expensive at scale
- Goal: train a small open-source model (1.5B params) to match larger models

Challenge

- 7 task categories ranging from simple queries to complex conflict resolution
- Must handle: tool sequencing, date math, conflict detection, vague language
- All training and inference on a single consumer GPU (12 GB VRAM)

System Architecture



get_time • list • get • create • update • delete • respond

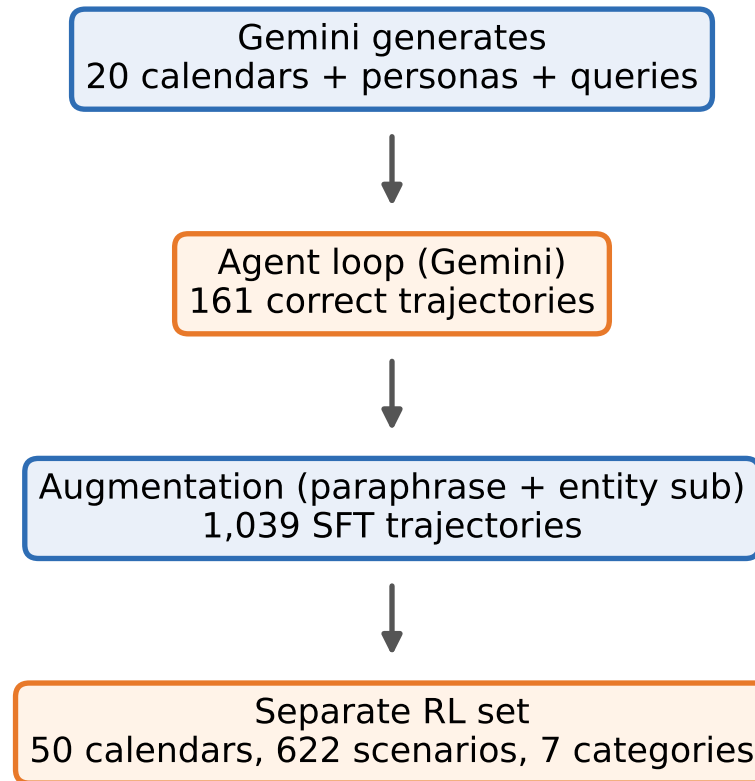
The 7 Calendar Tools

Tool	Required Params	Optional Params	Description
get_current_time	(none)	(none)	Get simulated date, time, day
list_events	(none)	time_min, time_max	List events, optionally filtered
get_event	event_id	(none)	Get full event details by ID
create_event	summary, start, end	description, attendees	Create a new event
update_event	event_id	summary, start, end, description	Update event fields
delete_event	event_id	(none)	Delete an event by ID
respond_to_event	event_id, attending	(none)	RSVP: ACCEPT/DECLINE/MAYBE

The 7 Task Categories

#	Category	Example Query	Difficulty
1	Relative Time References	"What's on my calendar tomorrow?"	Easy
2	Information Retrieval	"When is my next meeting with Alice?"	Easy
3	Schedule a Single Event	"Book a team standup at 9am Monday"	Medium
4	Vague & Contextual	"I need time to prepare for the board meeting"	Medium
5	Modifier & Correction	"Move my 3pm to 4pm and add Bob"	Hard
6	Human Chaos	"ugh cancel that thing tmrw morning"	Hard
7	Complex Logic & Conflict	"Find 1hr when Alice and Bob are both free"	Very Hard

Data Pipeline Overview



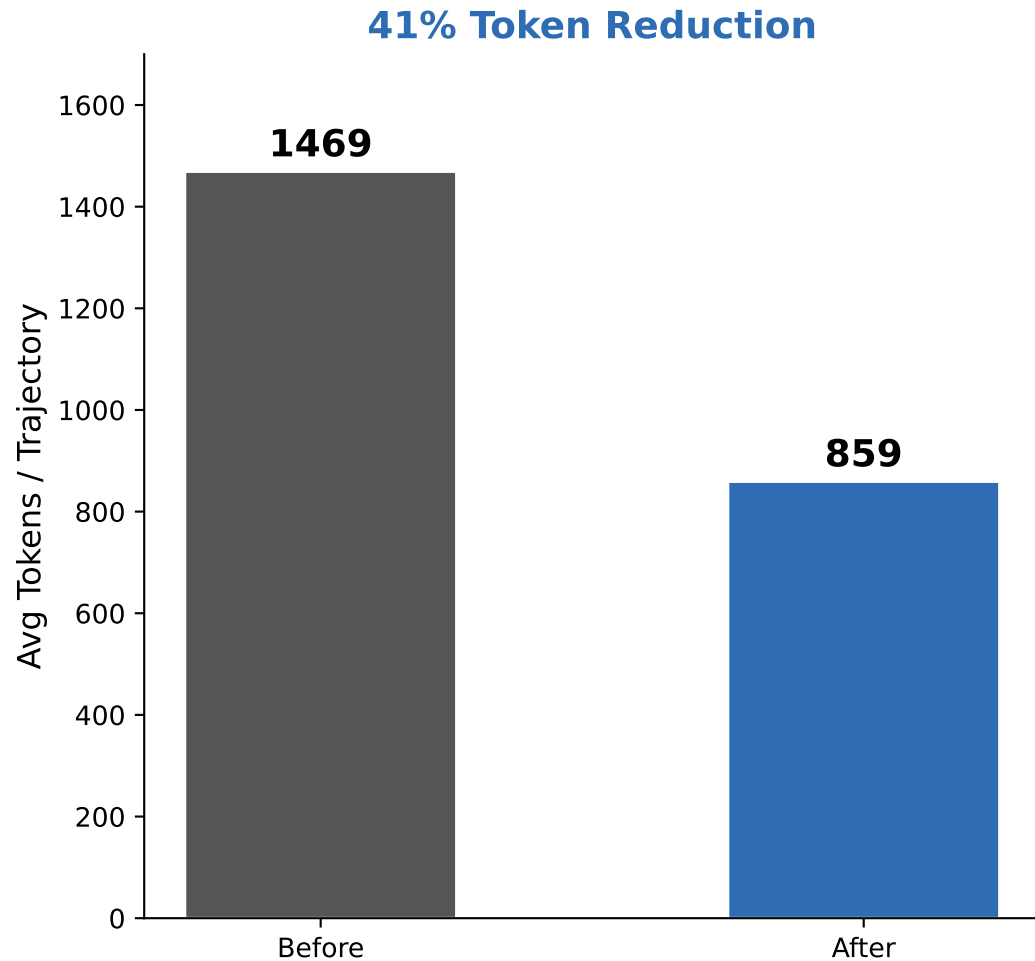
Data Generation — Gemini-Driven Pipeline

- 1 20 professions → Gemini → unique personas
- 2 Personas → Gemini → realistic weekly calendars (Mon-Sun)
- 3 Text calendars → Gemini → structured JSON with events
- 4 Calendars + dates → Gemini → categorized queries
+ expected behavior
- 5 Queries → Gemini agent loop → trajectories
(judge filters to "Correct" only)
- 6 Augmentation: 2 entity-sub variants
+ 3-10 paraphrases per trajectory

Key Numbers

50	calendars generated
161	correct trajectories
6-13×	expansion via augmentation
1,039	total SFT trajectories
280	RL evaluation scenarios
622	RL training scenarios

Data Engineering — Compact Format



- Flatten attendees (remove {"email": ...} wrappers)
- Strip verbose fields (creator, organizer, htmlLink, etc.)
- Native dicts instead of double-serialized JSON
- 159/161 trajectories fit in 3,076 token window

SFT Training Setup

Base Model

- Qwen2.5-1.5B-Instruct (open-source, tool-calling capable)

LoRA Configuration

- Rank 64, targeting q/k/v/o/gate/up/down projections (7 layers)

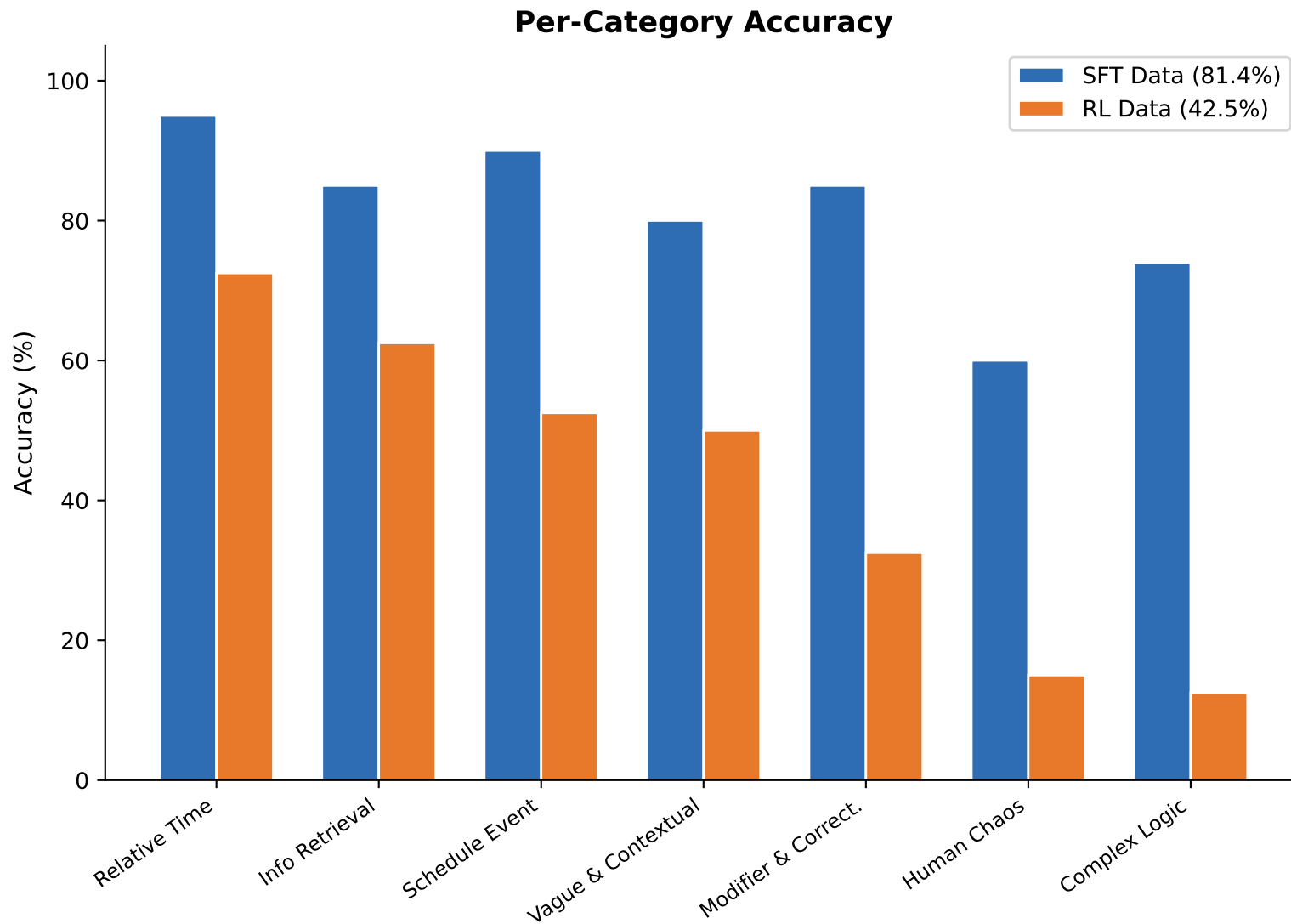
Training Details

- 1,039 trajectories, 10 epochs, cosine_with_restarts LR schedule
- Loss masking: only train on assistant tokens (tool calls + final answers)

Hardware Constraints

- TITAN X Pascal — fp16 only (no bfloat16, no FlashAttention2)
- Full SFT training completes in ~30 minutes

SFT Results (Best Checkpoint: Step 933)



Key Takeaway

SFT excels at
pattern-matching tasks

Struggles on
reasoning-heavy
categories

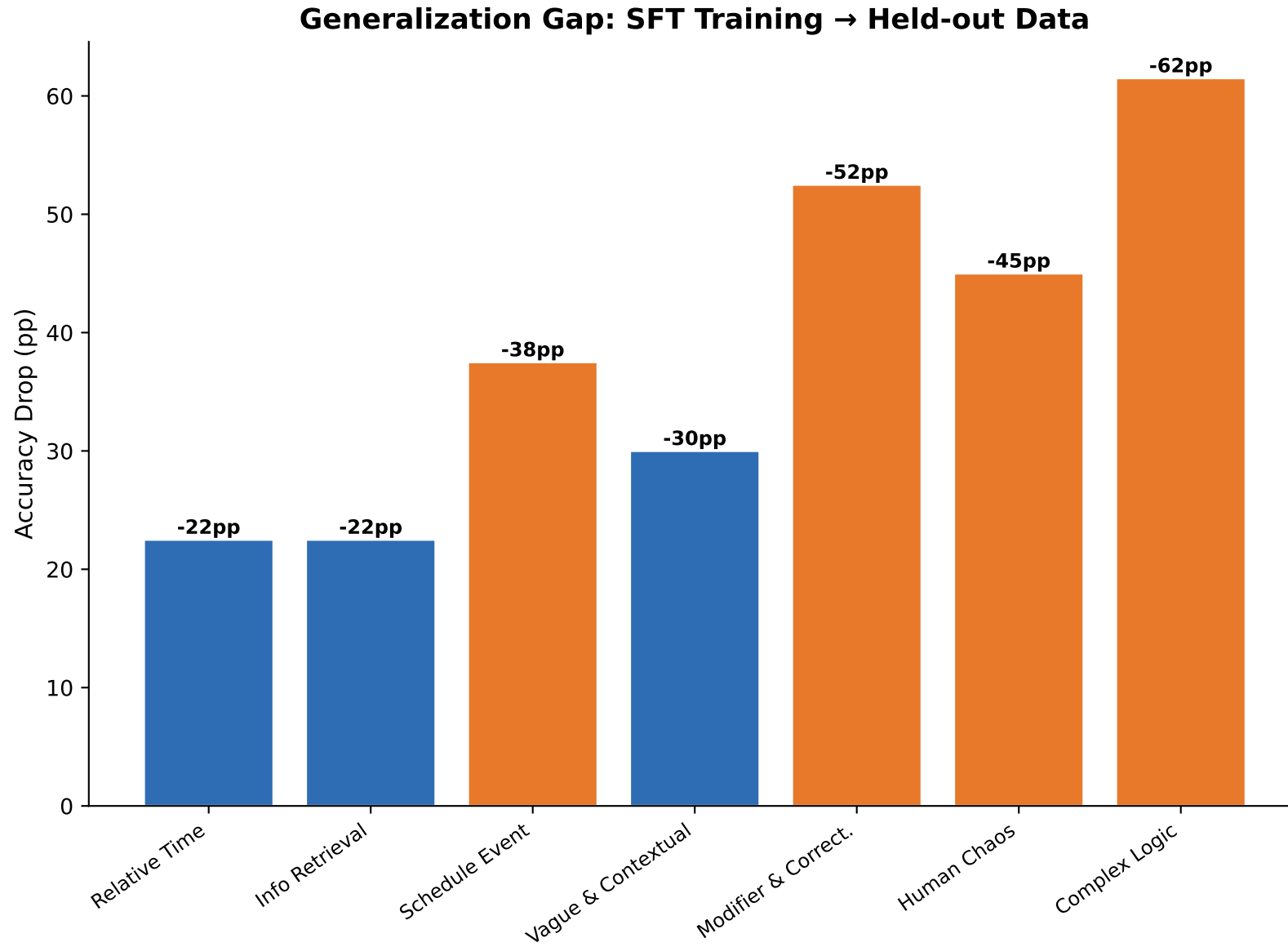
81.4%

on training data

42.5%

on held-out data

Why RL? The Generalization Gap



RL Hypothesis

Reward signal from
correct/incorrect
outcomes teaches
reasoning,
not just
pattern matching

RL Infrastructure — The Hard Part

Framework: ART 0.5.4 (OpenPipe) + Unsloth + GRPO

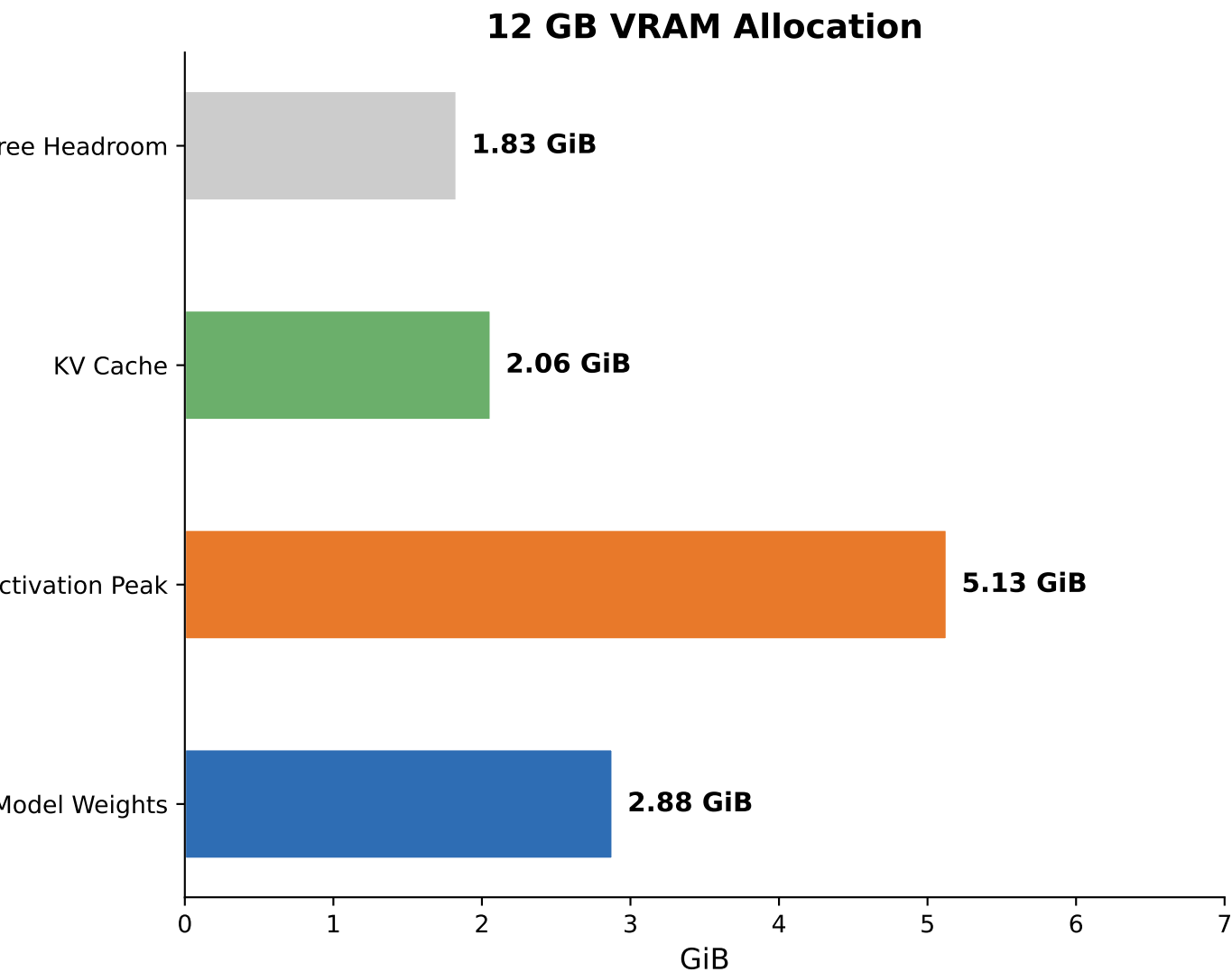
5 Critical Bugs Found & Patched

- Deadlock: asyncio/threading mismatch between ART service and HF Trainer
- OOM: health monitor timeout cascading to duplicate model loads
- Thread safety: asyncio.Queue used in synchronous context
- Autograd: entropy attached to computation graph unnecessarily
- Cache destruction: done_callback removing service on transient errors

All patches in `art_patches.py` — zero manual site-packages edits

- Reproducible monkey-patches applied at import time
- Stock ART 0.5.4 + our patches = stable training

GPU Memory Management — 12 GB Budget



Constraints Discovered

- LoRA rank 64 → OOM; capped at 32
- `gpu_memory_utilization` > 0.85 → OOM
- `batch_size` > 1 → OOM
- No FlashAttention2 (compute 6.1)
- No bfloat16 support
- ~1.9 GiB free headroom total

RL Training — Approach Evolution

✗ Attempt 1

Mixed Categories (all 7)

59% skip rate, no improvement

✗ Attempt 2

Curriculum Learning

+0.006 after 266 steps

⚠ Root Cause

GRPO needs multiple rollouts of same query per group

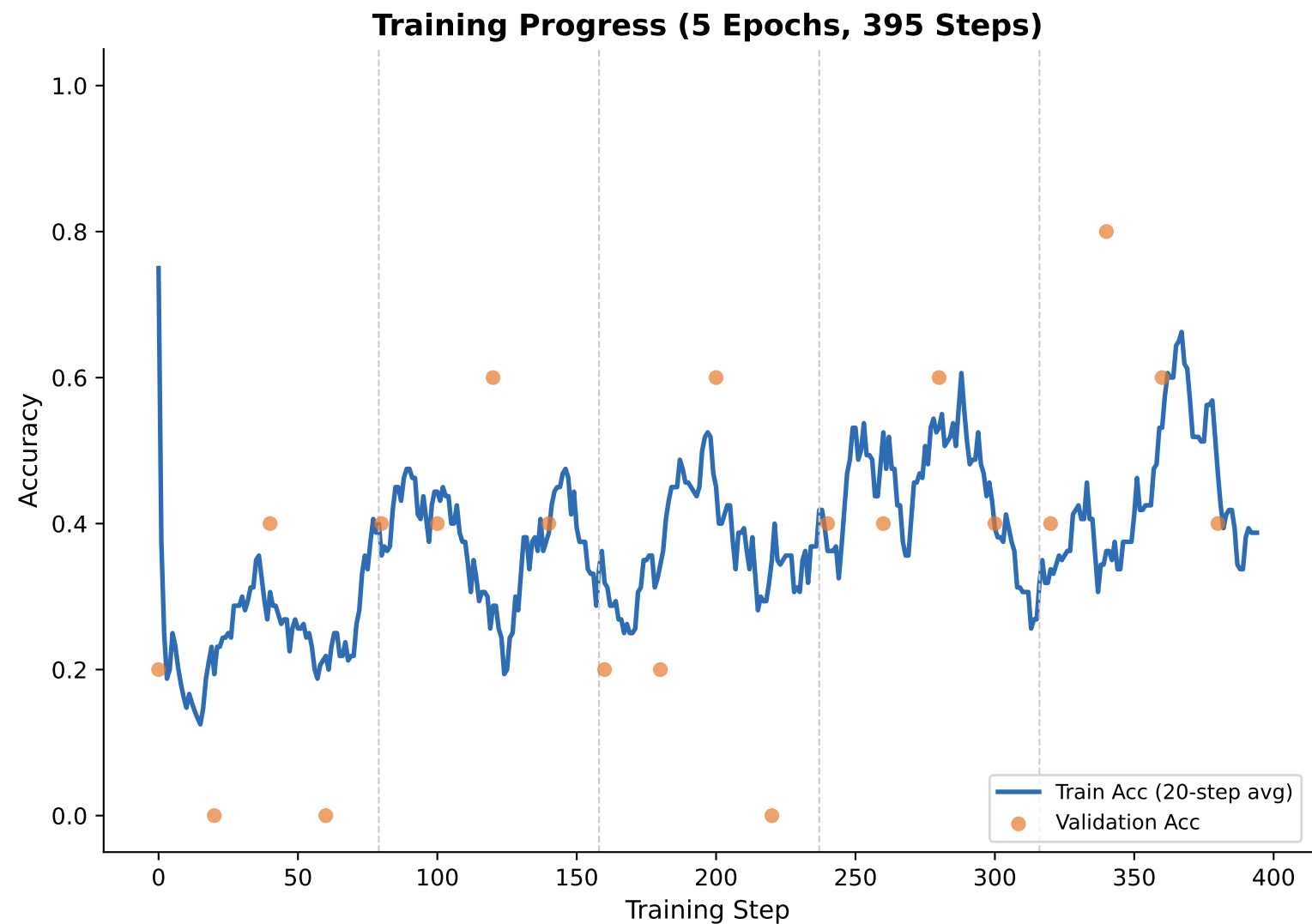
Mixed categories → unique queries
→ no reward contrast

✓ Attempt 3

Single-Category
(Modifier & Correction)

Proper grouping → success!

RL Training Results — Learning Curves



Epoch Summary

Epoch 0: 27.5% avg train

Epoch 1: 34.8% avg train

Epoch 2: 39.1% avg train

Epoch 3: 43.2% avg train

Epoch 4: 43.8% avg train

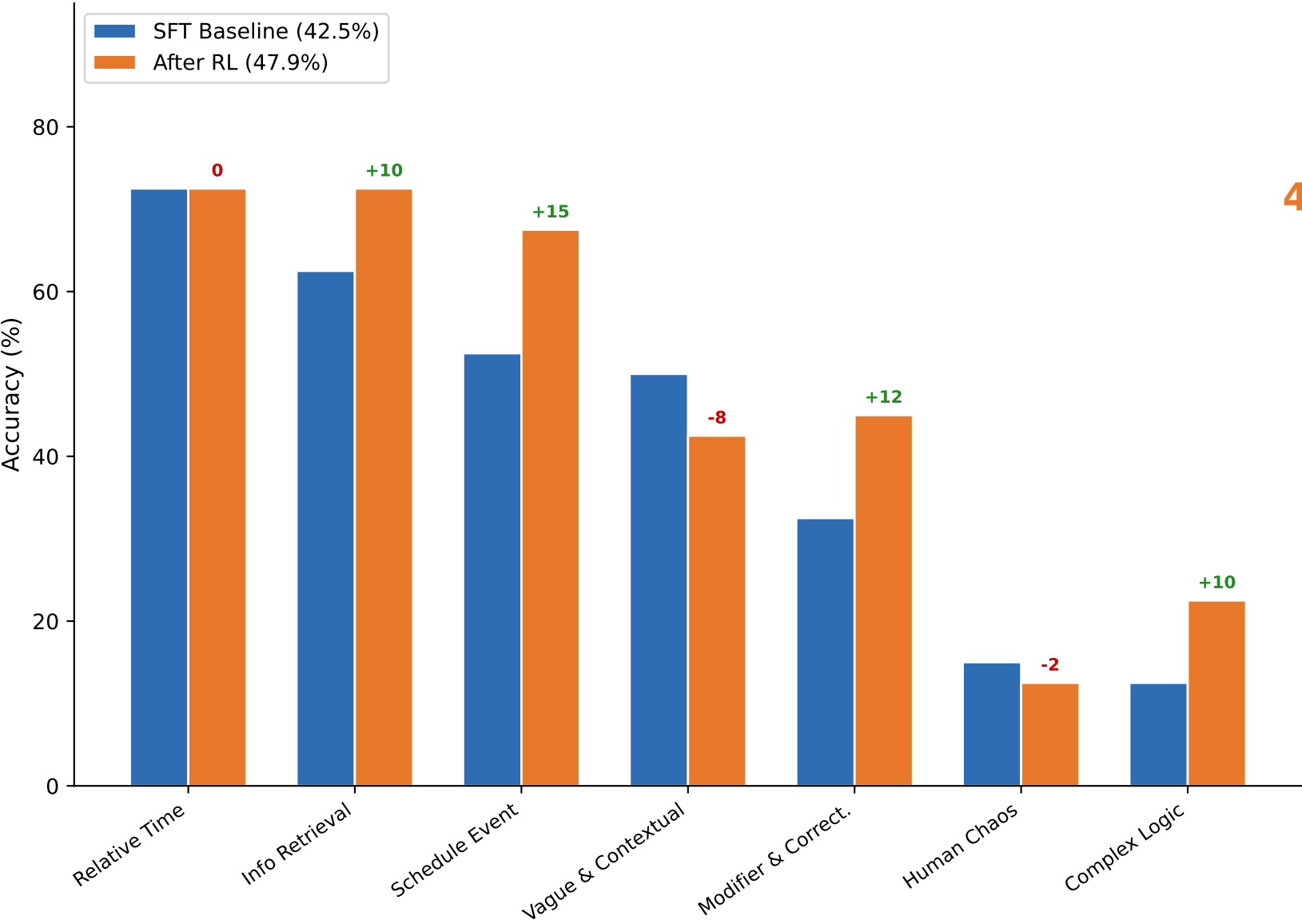
Start: 27.5%

End: 43.8%

RL Evaluation — Full Benchmark

Trained on Modifier & Correction only → tested on ALL 7 categories

Per-Category: SFT vs RL



Overall

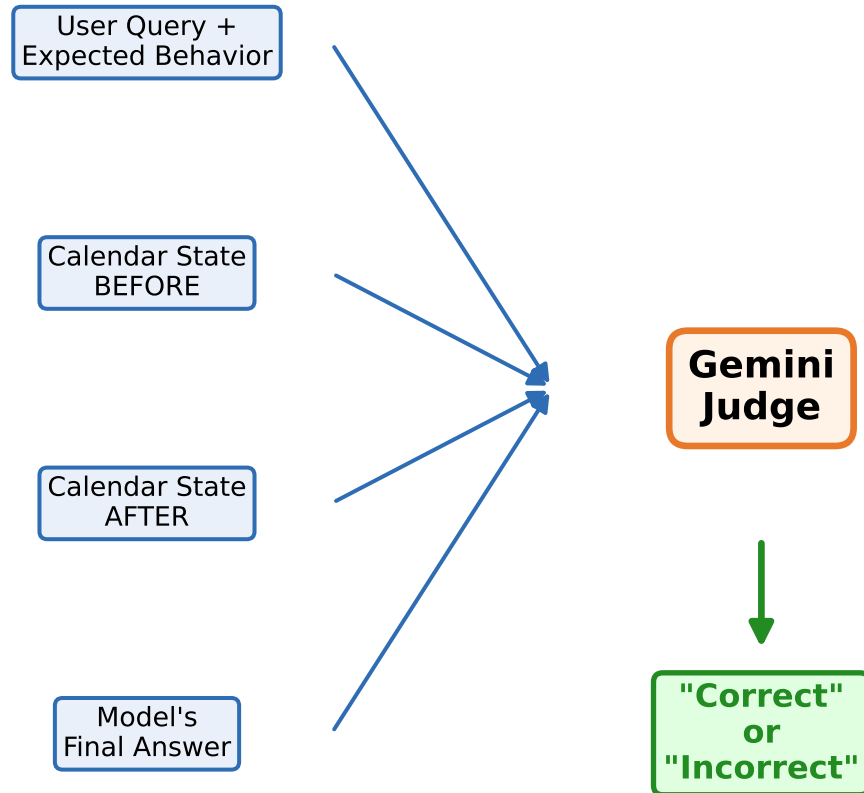
42.5% → 47.9%

+5.4 pp

Positive Transfer

- Schedule: +15 pp
- Info Retrieval: +10 pp
- Complex Logic: +10 pp

Automated Evaluation — Gemini as Judge



Judge Rules

- Must use tools before responding
- Action tasks: AFTER state must reflect expected changes
- Info tasks: response must match calendar data
- Partial completion = Incorrect
- Output: "Correct" or "Incorrect"

Same judge used for SFT data filtering, RL reward signal, and final evaluation

Key Technical Achievements

- End-to-end pipeline: data gen → SFT → RL → eval on a single consumer GPU
- 5 framework bugs found & fixed (ART/Unsloth) as reproducible patches
- 41% token reduction via compact event format engineering
- Positive transfer: single-category RL improved untrained categories
- Automated evaluation pipeline with Gemini as judge model
- ~6,500 LOC across 28 files, well-structured with tests
- Complete reproducibility: all configs, patches, and data tracked in repo

What's Holding Us Back — Hardware Limitations

Spec	TITAN X Pascal	A100	H100
VRAM	12 GB	80 GB	80 GB
bfloat16	No	Yes	Yes
FlashAttention	No	Yes	Yes
Compute Cap.	6.1	8.0	9.0
Est. Batch Size	1	16+	32+

Current Bottlenecks:

- batch_size=1, num_generations=2 (vs 8-16 typical)
- max_lora_rank=32 (vs 128+ typical)
- 3,076 token context (vs 8K+ typical)
- Single category at a time (can't do multi-category RL)
- ~2 hours per epoch of 79 scenarios

What Better Hardware Unlocks

With 80 GB GPU (A100 / H100)

- Multi-category RL training (all 7 categories simultaneously)
- Larger batch sizes → faster convergence, better reward contrast
- Larger context window (8K+) → handle complex calendars
- Higher LoRA rank (128+) → more expressive adaptation
- FlashAttention2 → 2-3x faster inference during rollouts

Scale to Larger Models

- Qwen2.5-7B or 14B → much higher performance ceiling
- Multi-GPU setups enable full fine-tuning, not just LoRA

Projected Impact

- Estimated: 60-70%+ on full benchmark (vs 47.9% current)

Summary & Next Steps

What We Built

- ✓ Complete agentic AI training pipeline from scratch
- ✓ Data generation → SFT → RL → automated evaluation
- ✓ All on a single consumer-grade GPU (12 GB)

What We Achieved

- ✓ 47.9% on challenging 7-category benchmark (+5.4pp from RL)
- ✓ Positive transfer: RL on 1 category improved 3 others
- ✓ 5 open-source framework bugs discovered and patched

Next Steps (with better hardware)

1. Multi-category RL across all 7 categories
2. Scale to Qwen2.5-7B base model
3. Dynamic difficulty grouping for hard categories
4. Target: 70%+ overall accuracy